

Государственное бюджетное общеобразовательное учреждение
Самарской области средняя общеобразовательная школа №8
«Образовательный центр» имени В. З. Михельсона города Новокуйбышевска
городского округа Новокуйбышевск Самарской области

Тема: Альтернативные QR-кода. Разработка концепции CF-кодов

Автор:

Туйзюков Артур,
ученик 10 «А» класса ГБОУ СОШ № 8 «ОЦ»

Руководитель:

Бобкова Анастасия Алексеевна,
учитель информатики

г.о. Новокуйбышевск, 2024 год

АННОТАЦИЯ

В настоящее время QR-кода получили широкое распространение во всем мире, но несмотря на это, они представляют собой невзрачный набор черно-белых квадратов, зачастую портящих эстетическую часть предмета. Изменить внешний облик QR-кодов и сделать их более благоприятными способны различные альтернативные QR-кода, в соответствии с чем реализована концепция CF-кодов в рамках телеграм бота, позволяющего считывать и различными способами создавать CF-кода.

КЛЮЧЕВЫЕ СЛОВА

QR-КОДА, CF-КОДА, ТЕЛЕГРАМ БОТ, СЧИТЫВАНИЕ, ГЕНЕРАЦИЯ, ПИКСЕЛИЗАЦИЯ, БАЗА ДАННЫХ, ИЗОБРАЖЕНИЯ

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ [3]

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ [4]

1.1 QR-код и история его создания [4]

1.2 Принцип работы QR-кода [5]

1.3 Альтернативные вариации QR-кодов [6]

1.4 Внешнее устройство CF-кода [7]

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ [9]

2.1 Принцип работы CF-кодов [9]

2.2 Написание файла для работы с CF-кодами [9]

2.3 Написание файла для работы с базой данных [12]

2.4 Написание файла для работы с телеграм ботом [12]

ЗАКЛЮЧЕНИЕ [15]

ПРИЛОЖЕНИЕ 1 [16]

ПРИЛОЖЕНИЕ 2 [17]

ПРИЛОЖЕНИЕ 3 [22]

ПРИЛОЖЕНИЕ 4 [25]

БИБЛИОГРАФИЧЕСКИЙ СПИСОК [40]

ВВЕДЕНИЕ

Актуальность выбранной темы: в настоящее время QR-кода получили большое распространение и активное использование во всем мире, в связи с чем рассмотрение альтернативных QR-кодов является актуальной темой.

Цель проекта: рассмотрение возможности применения альтернативных QR-кодов, разработка концепции CF-кодов.

Задачи проекта:

- изучение принципов работы QR-кодов;
- рассмотрение возможности применения альтернативных qr-кодов;
- разработка концепции CF-кодов;

Объект исследования: QR-кода.

Предмет исследования: альтернативные QR-кода.

Методы исследования:

Эмпирический метод исследования – поиск литературы по данной теме.

Теоретический метод исследования – расчет способов достижения необходимой задачи.

Практико-ориентированный метод – написание программного кода.

Теоретическая значимость: рассмотрение возможности применения альтернативных QR-кодов.

Практическая значимость: разработка концепции CF-кодов.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 QR-код и история его создания

QR-код (в переводе с английского Quick Response code - код быстрого реагирования) — двумерный тип штриховых кодов, который легко считывается цифровым устройством и хранит информацию в виде серии пикселей в квадратной сетке, которая внешне выглядит как черно-белый узор.

Свое начало QR-код берет в 1960-х годах, когда Япония вступила в период быстрого экономического роста и во многих районах страны начали появляться супермаркеты, торгующие широким спектром товаров. Кассирам приходилось вручную вбивать их стоимость на кассовых аппаратах, из-за чего у многих работников развивался синдром запястного канала. Тогда была разработана POS-система, в которой цена товара автоматически отображалась на кассовом аппарате, когда его штрих-код сканировал оптический датчик. Но по мере распространения штрих-кодов стали очевидны и их ограничения. Самым заметным из них был тот факт, что такой код может содержать не более 20 цифро-буквенных символов.

В начале 1990-х годов японская Denso Wave Inc., дочерняя структура Toyota, которая занималась разработкой считывателей штрих-кодов, решила создать новые коды, которые могли бы содержать больше информации. Чтобы ускорить чтение кода, разработчики добавили угловые метки, которые сначала видит считыватель. Затем они изучили соотношение белых и черных областей на изображениях и символах в листовках, журналах, на картонных коробках и прочей продукции. Исследователи выбрали самое непопулярное соотношение, которое не использовалось в печатной продукции и в штрих-кодах и при этом могло считываться под любым углом.

В 1994 году был представлен первый QR-код. Он мог кодировать до 7 тыс. знаков и считывался в 10 раз быстрее, чем штрих-код. Первой такие коды стала использовать автомобильная промышленность. Они значительно упростили все управленческие задачи от контроля производства до доставки и выдачи квитанций о транзакциях.

Denso Wave открыла свою разработку для свободного использования и в 2000 году Международная организация по стандартизации внесла QR-код в список одобренных стандартов кодирования информации.

1.2 Принцип работы QR-кода

Узор QR-кода хранит зашифрованную последовательность данных в двоичном формате (1 и 0) в виде матрицы. Каждой отдельной ячейке сетки присваивается значение в зависимости от цвета (черный или белый). Затем ячейки группируются в более крупные узоры. Ключи закодированных данных содержат дубликаты, поэтому при повреждении поверхности QR-кода до определенных масштабов его можно считать.

QR-код имеет определенное устройство (см. рис. 1), разбивающее его на несколько частей. Сканер распознает QR-код по трем квадратным меткам (поисковый узор, на рис. 1 отмечен красным), расположенным по его углам. Они указывают, в каком направлении читать код. Обнаружив их, сканер считывает содержание квадрата, а затем анализирует QR-код, представляя его в виде сетки. Процесс считывания обеспечивает специализированное программное обеспечение, способное извлекать данные из шаблонов в матрице.



Рис 1. Устройство QR-кода

Также каждый QR-код имеет полосы синхронизации (на рис. 1 отмечены голубым) и выравнивающий узор (на рис. 1 отмечен фиолетовым), чтобы его можно было считать даже на неровной поверхности.

Кроме того, QR-код включает маркер его версии (на рис. 1 отмечен синим), то есть сведения о формате (на рис. 1 отмечены желтым), в котором закодированы данные. Всего их четыре: цифровое, буквенно-цифровое, двоичное и кандзи для японских иероглифов.

QR-код имеет также блоки исправления ошибок Рида — Соломона, которые располагаются по краям. Коды Рида-Соломона представляют собой специальную группу кодов, исправляющих ошибки при чтении. Таким образом, даже при повреждении 30% поверхности QR-кода, сканер считывает его правильно.

Наконец, каждый QR-код отделяется от внешнего пространства белым пространством или «тихой зоной» (на рис. 1 отмечена зеленым), она нужна, чтобы сканер распознал код.

1.3 Альтернативные вариации QR-кодов

В рамках вариаций QR-кодов, разработанных компанией Denso Wave, принципиальных отличий от стандартной модели нет. Единственные различия заключаются в размерах QR-кода и наличии посередине фотографии (frame QR), в целом не влияющие на внешний вид.

Существуют и другие вариации QR-кода, имеющие отличия от стандартного. Поисковый узор Aztec кода находится посередине в отличие от стандартного расположения по углам. MaxiCode схож с Aztec кодом, но его поисковый узор круглый, а сами черно-белые квадраты, несущие информацию, заменены правильными шестиугольниками. PDF417 представляет собой смесь QR и штрих кодов. Data Matrix код представляет собой упрощенную версию стандартного QR-кода, отличающуюся отсутствием поисковых узоров.

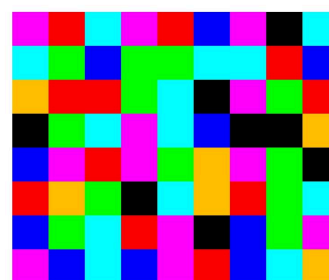
Стоит отметить, что использование любого другого цвета вместо черного, не меняет структуры QR-кода и не имеет никаких других отличий, кроме замены черного цвета. По-настоящему разукрасить QR-код удалось ученым из университета Бразилии. Вместо использования двух цветов (черного и белого) им удалось создать работающую модель CQR-кода (colored quick-response), использующую четыре цвета: красный, зеленый, синий и белый. Но данный прототип все так же оставался мало привлекательным, имея общую структуру с QR-кодом, и не мог нести в себе осмысленные изображения.

С целью эстетического изменения внешнего вида QR-кода, позволяющего придать ему красочность и разнообразие, вместо стандартных черно-белых квадратов, можно использовать различные символы (например #0X=%\$ или другие узоры) и даже создавать из них ASCII изображения или аналогично CQR-коду придать более приятный вид за счет не только использования различных цветов вместо черно-белого варианта, но и создания различных осмысленных изображений.

В рамках данного проекта остановимся на создании цветного CF (colourful) кода имеющего схожую задумку с CQR-кодом, но отличающегося от него внешним устройством, не схожим с QR-кодом, и использованием семи различных цветов, вместо четырех, с возможностью создавать осмысленные изображения.

1.4 Внешнее устройство CF-кода

CF-код (см рис 1) подобно QR-коду представляет собой прямоугольник с ячеистым узором, но информация в нем кодируется не в двоичном, а в семиричном формате, поскольку для кодирования используется семь различных цветов: черный, красный, желтый, зеленый, циан, синий и розово-фиолетовый. Выбраны именно эти цвета так как они однозначно раскладываются на цветовую палитру rgb, в связи с чем легко распознаются и не могут быть между собой перепутаны.



9 x 8

Рис 2. CF-код

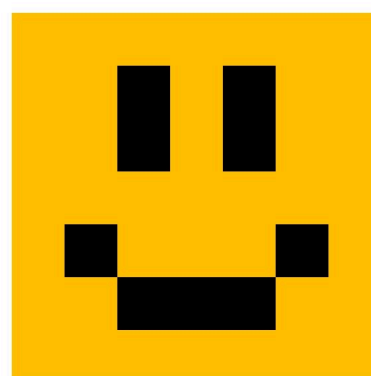
Поскольку CF-код может иметь любой размер, а для его считывания необходимо знать размерность, то для удобства под CF-кодом располагаются два числа, разделенных знаком “х”, указывающие его размеры.

CF-код не имеет служебных элементов в своем узоре, все ячейки в нем используются для кодирования, из-за чего при повреждении не может быть считан, но позволяет нести в себе осмысленные изображения (см рис 3 и 4), что и является главным эстетическим преимуществом перед QR-кодом.



17 x 14

Рис 3. CF-код в виде Марио



7 x 7

Рис 4. CF-код в виде смайлика

ВЫВОДЫ ПО 1 ГЛАВЕ

В процессе изучения литературы по данной теме были изучены методы работы QR-кодов, их альтернативные вариации, рассмотрен принцип внешнего устройства CF-кода.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Принцип работы CF-кодов

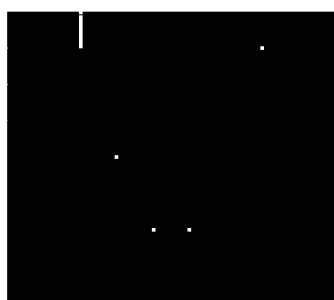
Для удобства использования создадим концепцию CF-кодов в виде телеграм бота. Для написания кода будем использовать высокоуровневый язык python, поскольку он активно поддерживается, предоставляет доступ к различным библиотекам (список используемых библиотек и их версии см в приложении 1). В качестве среды разработки будет использоваться приложение PyCharm, позволяющее вести работу с программами, написанными на языке python.

Телеграм бот должен иметь следующие функции: считывание CF-кода, генерация случайного, загрузка готового, создание из изображения путем пикселизации, создание собственного, просмотр созданных пользователем CF-кодов, их изменение и удаление. Для удобства разработки разобьем весь необходимый код на три отдельных файла, выполняющих определенный спектр задач: файл cfcode.py для работы с CF-кодами, файл database.py для работы с базой данных и файл tgbot.py для создания телеграм бота. Файлы взаимосвязаны между собой следующим образом: пользователь взаимодействует с телеграм ботом написанным в файле tgbot.py, откуда посылается запрос в другие файлы database.py или cfcode.py в соответствии с нужной командой.

Создать своего телеграм бота можно с помощью официального бота BotFather, там же можно получить токен, необходимый для функционирования написанного кода. Разработку начнем с создания всех необходимых функций для считывания и создания CF-кодов в соответствующем файле. Для работы с CF-кодами нам понадобятся библиотеки opencv, pillow и numpy.

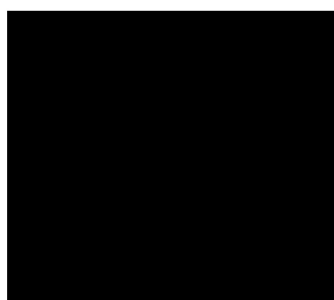
2.2 Написания файла для работы с CF-кодами

Для считывания CF-кода создадим функцию detect_code с использованием функций библиотеки opencv, принимающую три параметра: изображение для считывания (см рис 2), количество столбцов и строк CF-кода. Поскольку CF-код имеет белую окантовку, то для поиска границ CF-кода воспользуемся методом бинаризации изображения (см рис 5) с помощью функции inRange, поскольку белый цвет хорошо различается с используемыми для кодирования цветами. Уберем на изображении получившиеся шумы (см рис 6) с помощью функции erode.



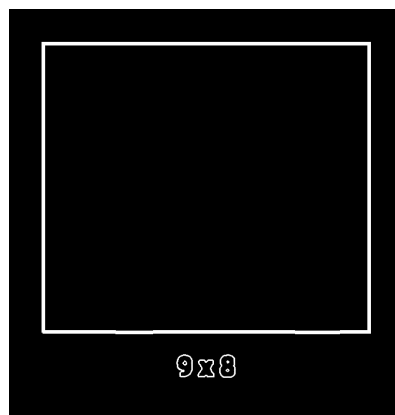
9 x 8

Рис 5. Бинаризация изображения



9 x 8

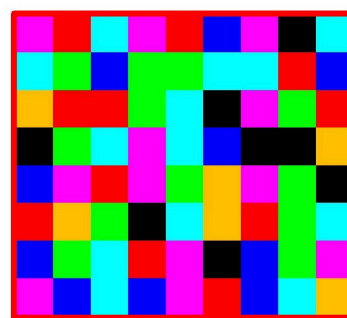
Рис 6. Очистка изображения от шумов



9 x 8

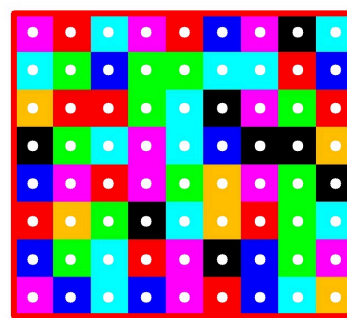
Рис 7. Поиск контуров на изображении

Используя функции `Canny` и `minAreaRect`, найдем контуры на полученном изображении (см рис 7) и сведем их к прямоугольникам, задающимся четырьмя опорными точками. Из найденных прямоугольников выберем наибольший (см рис 8) и проанализируем цвета стоящие в узлах сетки (см рис 9), соединяющей центры ячеек узора (сетка проходится циклом в соответствии с переданными размерами кода). Нужный пиксель изображения разложим на 3 составляющие цветовой палитры `rgb` (см рис 10) функцией `split` библиотеки `opencv` и с помощью функции `thresh` той же библиотеки конкретизируем значение данного пикселя. В соответствии с полученными значениями `rgb` определим цвет и запишем в переменную его номер в соответствии с определенными нами номерами для каждого цвета (черный - 0, красный - 1, желтый - 2, зеленый - 3, циан - 4, синий - 5, розово-фиолетовый - 6). Так как CF-код может иметь большой размер, то для укорочения полученное число из последовательности переведем из 7-ричной системы счисления в 64-ричную, с помощью отдельно написанной функции `convert_to`. В результате функция возвращает 64-ричное распознанное число и размерность CF-кода.



9 x 8

Рис 8. Границы CF-кода



9 x 8

Рис 9. Узлы сетки

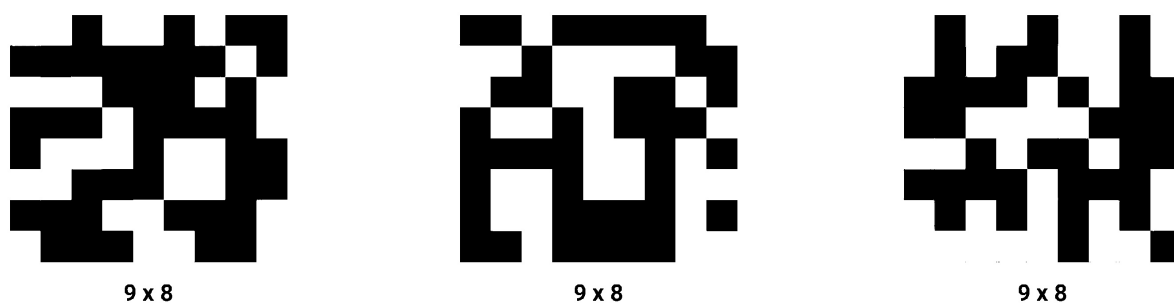


Рис 10. Разложение изображения на красный, зеленый и синий спектры

Для пикселизации изображения (см рис 11) создадим функцию `pixelate`, принимающую два параметра: изображение и уровень пикселизации. Уменьшим размеры изображения и тем самым пикселизируем его с помощью метода `resize` с параметром `Image.NEAREST` библиотеки `pillow`. Поскольку цвета пикселей полученного изображения отличаются от нужных нам, то для каждого пикселя аналогично функции `detect_code` классифицируем цвет в соответствии с его значением в палитре `rgb`, получим последовательность номеров цветов и конвертируем ее в 64-ричную систему счисления. В результате функция возвращает 64-ричное число и размерность CF-кода.

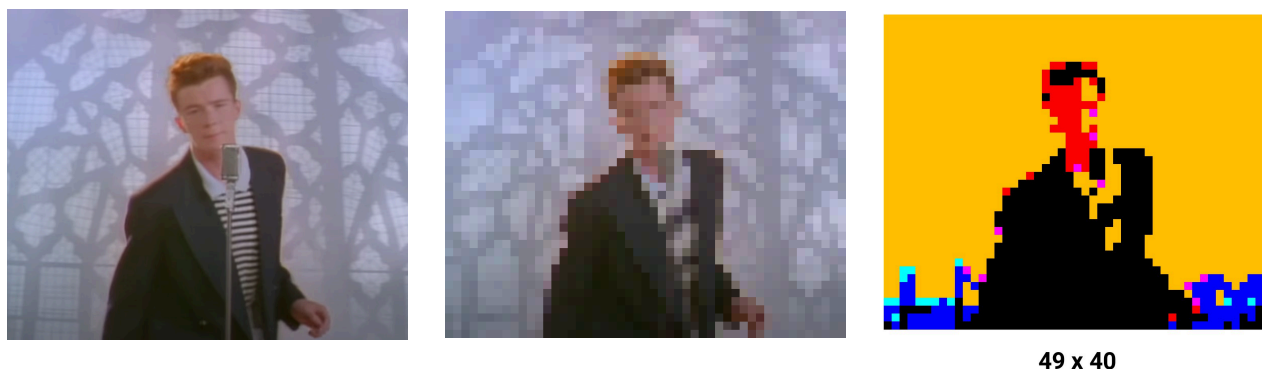


Рис 11. Пример работы пикселизации изображения

Для создания CF-кода из числа создадим функцию `generate_code`, принимающую 4 обязательных параметра: число, количество строк и столбцов CF-кода, наличие границ ячеек и 4 необязательных параметра: наличие промежутков между ячейками, телеграм `id` пользователя, необходимость конвертирования числа, уровень пикселизации. С помощью методов `Image.new` и `ImageDraw.Draw` создадим новое изображение и, пройдя циклом по переданному числу, закрасим нужные участки изображения в соответствии с номером

цвета. Под CF-кодом с помощью метода `text` укажем размеры CF-кода. Сохраним изображение и возвратим нужные значения в соответствии с передаваемыми значениями.

2.3 Написания файла для работы с базой данных

Для правильного функционирования телеграм бота, нам понадобится база данных, хранящая всю информацию о пользователях и созданных ими CF-кодов. Для создания базы данных будем использовать встроенную библиотеку `sqlite3`. Создадим базу данных и курсор для работы с ней с помощью функций `connect` и `cursor`. В базе данных информация хранится в двух таблицах: таблице `users`, хранящей всю необходимую информацию о пользователе, и таблице `cfcodes`, хранящей информацию о созданных CF-кодах. Посредством написания различных SQL-запросов и обращения в базу данных с помощью курсора создадим все необходимые функции для добавления, изменения и удаления информации.

2.4 Написания файла для работы с телеграм ботом

В качестве библиотеки для написания телеграм бота выбрана библиотека `aiogram`. С помощью функций `Bot` и `Dispatcher` запустим телеграм бота, передав его токен. Создадим всю необходимую логику работы бота, используя обработчики событий `message_handler` с различными параметрами в зависимости от типа обрабатываемого сообщения. Помимо стандартных команд для навигации бота, стоит выделить элементы кода, касающиеся CF-кодов.

Для считывания CF-кода пользователь нажимает кнопку “Считать CF-код”, указывает его размеры, после чего вызывается функция `detect_code`, принцип работы которой был описан ранее. Полученное число отправляется вместе с запросом в базу данных файла `database.py`, откуда возвращается информация о считанном CF-коде, если такой существует.

Для загрузки готового CF-кода нажимает кнопку “Загрузить готовое”, где используется алгоритм аналогичный считыванию, но в отличие от считывания полученное число отправляется с запросом в базу данных для добавления нового CF-кода, а не получения информации об уже существующем.

Для создания CF-кода путем пикселизации изображения пользователь нажимает кнопку “Пикселизировать”, вводит текст, который будет храниться в коде, и отправляет

изображение для пикселизации. Изображение пикселизируется тремя различными уровнями (16, 24 и 32) используя функцию `pixelate`. Выбранный пользователем уровень пикселизации сохраняется в базу данных.

Для генерации CF-кода, пользователь нажимает кнопку “Сгенерировать случайный” и вводит текст, который будет храниться в коде. Генерация CF-кода начинается со случайного выбора размера кода с помощью функции `randint` библиотеки `random`. Ширина выбирается случайным числом от 3 до 10, а высота случайным числом от 2 до значения высоты - 1. Используя функцию `choices`, создается число в 7-ричной системе счисления нужной длины. Далее полученное число передается одним из параметров функции `generate_code`, написанной нами в файле `cfcode.py`, где создается изображение CF-кода. В базу данных `database.py` так же посылается запрос, на добавление нового кода.

Для создания собственного CF-кода пользователь нажимает кнопку “Создать CF-код” и вводит текст, который будет храниться в коде. Далее указывается количество строк и столбцов CF-кода и с помощью функции `generate_code` создается пустой CF-код с рамками ячеек, которые постепенно заполняет пользователь. Полученный в конце код сохраняется и добавляется в базу данных.

ВЫВОДЫ ПО 2 ГЛАВЕ

В процессе работы был написан программный код на языке программирования python для считывания, генерации CF-кодов, созданный в рамках телеграм бота. Создана база данных для корректной работы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данного проекта были приобретены навыки работы в следующих сферах: программирование на языке python с использованием библиотеки машинного зрения opencv, библиотеки для работы с телеграм ботом aiogram и создания изображений pillow. Была создана концепция CF-кодов в рамках функционирующего телеграм бота с базой данных.

ПРИЛОЖЕНИЕ 1

Список используемых библиотек python версии 3.11.3:

1. aiogram (версия 2.25.2)
2. numpy (версия 1.26.2)
3. opencv_contrib_python (версия 4.8.1.78)
4. Pillow (версия 10.1.0)
5. Requests (версия 2.31.0)

ПРИЛОЖЕНИЕ 2

Полный код файла cfcode.py:

```
from PIL import Image, ImageDraw, ImageFont
import cv2 as cv
import numpy as np
import database as db

# Функция для перевода из одной системы счисления в другую
async def convert_to(n, base_from, base_to):
    alph =
'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ-_'
    m = 0
    for i in range(len(n) - 1, -1, -1):
        m += alph.find(n[i]) * base_from ** (len(n) - i - 1)
    result = ""
    while m != 0:
        result += alph[m % base_to]
        m //= base_to
    return result[::-1]

# Функция уменьшения размера изображения для удобства отображения при отладке
async def rescale_frame(frame, scale=0.5):
    width = int(frame.shape[1] * scale)
    height = int(frame.shape[0] * scale)
    return cv.resize(frame, (width, height), interpolation=cv.INTER_AREA)

# Функция пикселизации изображения
async def pixelate(image, pixel_size=16):
    image = image.resize((image.size[0] // pixel_size, image.size[1] // pixel_size),
Image.NEAREST)
    result = ""
    colors = {'0, 0, 0': 0, '0, 0, 255': 1, '0, 255, 255': 2, '0, 255, 0': 3, '255, 255, 0': 4, '255, 0, 0': 5,
'255, 0, 255': 6, '255, 255, 255': 2}
    image = np.array(image)
    r, g, b = cv.split(image)
    _, image_b = cv.threshold(b, 120, 255, cv.THRESH_BINARY)
    _, image_g = cv.threshold(g, 120, 255, cv.THRESH_BINARY)
    _, image_r = cv.threshold(r, 120, 255, cv.THRESH_BINARY)
    for i in range(0, image.shape[0]):
        for j in range(0, image.shape[1]):
```

```

        color = colors[f'{image_b[i, j]}, {image_g[i, j]}, {image_r[i, j]}']
        result += str(color)
    return await convert_to(result, 7, 64), image.shape[0], image.shape[1]

# Считывание CF-кода с изображения
async def detect_code(image, x, y):
    image = cv.cvtColor(np.array(image), cv.COLOR_RGB2BGR)
    min_p = (150, 150, 150)
    max_p = (255, 255, 255)
    image_g = cv.inRange(image, min_p, max_p)
    kernel = np.ones((5, 5), np.uint8)
    erosion = cv.erode(image_g, kernel, iterations=1)
    canny = cv.Canny(erosion, 125, 200)

    contours, hierarchy = cv.findContours(canny, cv.RETR_TREE,
cv.CHAIN_APPROX_SIMPLE)
    boxcf = []

    for contour in contours:
        rect = cv.minAreaRect(contour)
        area = int(rect[1][0] * rect[1][1])
        box = cv.boxPoints(rect)
        box = np.int0(box)

        if boxcf == []:
            boxcf = [box, area, rect]
        elif area > boxcf[1]:
            boxcf = [box, area, rect]

    box = boxcf[0]
    main_box = []
    #cv.drawContours(image, [box], 0, (0, 0, 255), 10)
    #cv.imshow('image', image)
    #cv.waitKey()

    for elem in box:
        main_box.append([int(elem[0]), int(elem[1])])
    main_box = sorted(main_box)

    if main_box[0][1] < main_box[1][1]:
        top_left = main_box[0]
        bottom_left = main_box[1]
    else:

```

```

    top_left = main_box[1]
    bottom_left = main_box[0]
    if main_box[2][1] < main_box[3][1]:
        top_right = main_box[2]
        bottom_right = main_box[3]
    else:
        top_right = main_box[3]
        bottom_right = main_box[2]

    result = "
    colors = {'0, 0, 0': 0, '0, 0, 255': 1, '0, 255, 255': 2, '0, 255, 0': 3, '255, 255, 0': 4, '255, 0, 0': 5,
'255, 0, 255': 6, '255, 255, 255': 7}
    colors_name = {0: 'black', 1: 'red', 2: 'yellow', 3: 'green', 4: 'cyan', 5: 'blue', 6: 'pink', 7: 'white'}
    for i in range(y):
        for j in range(x):
            coords = [round(top_left[0] + (top_right[0] - top_left[0]) / x * j + (top_right[0] -
top_left[0]) / x / 2 + (bottom_left[0] - top_left[0]) / y * i), round(top_left[1] + (bottom_left[1] -
top_left[1]) / y * i + (bottom_left[1] - top_left[1]) / y / 2 + (top_right[1] - top_left[1]) / x * j)]
            img = image[coords[1] - 3: coords[1] + 3, coords[0] - 3: coords[0] + 3]
            img = cv.blur(img, (10, 10))
            b, g, r = cv.split(img)
            _, img_b = cv.threshold(b, 120, 255, cv.THRESH_BINARY)
            _, img_g = cv.threshold(g, 120, 255, cv.THRESH_BINARY)
            _, img_r = cv.threshold(r, 120, 255, cv.THRESH_BINARY)
            color = colors['{img_b[img.shape[0] // 2, img.shape[1] // 2]}, {img_g[img.shape[0] // 2,
img.shape[1] // 2]}, {img_r[img.shape[0] // 2, img.shape[1] // 2]}']
            result += str(color)
    return await convert_to(result, 7, 64), y, x

```

Генерация CF-кода

```

async def generate_code(n, h, w, borders, gaps=False, user_id=None, convert=True, pixelate=0):
    image_shape = [h, w]
    if not borders and convert:
        n = await convert_to(n, 64, 7)
    if len(n) < h * w:
        n = '0' * (h * w - len(n)) + n

    if h * w < 100:
        square = 80
    if square * max(h, w) // 10 > 160:
        border = square * max(h, w) // 10
    else:
        border = 80

```

```

if square * h // 10 > 80:
    num = square * h // 10
else:
    num = 80
if gaps:
    grid_gap = 8
else:
    grid_gap = 0
else:
    square = 20
    if square * max(h, w) // 10 > 40:
        border = square * max(h, w) // 10
    else:
        border = 40
    if square * h // 10 > 20:
        num = square * h // 10
    else:
        num = 20
    if gaps:
        grid_gap = 2
    else:
        grid_gap = 0
num_gap = num // 2

colors = {'0': (0, 0, 0), '1': (0, 0, 255), '2': (0, 190, 255), '3': (0, 255, 0), '4': (255, 255, 0), '5':
(255, 0, 0), '6': (255, 0, 255), '7': (255, 255, 255)}
image = Image.new('RGB', (border * 2 + grid_gap * (image_shape[1] - 1) + square *
image_shape[1], border * 2 + grid_gap * (image_shape[0] - 1) + square * image_shape[0] +
num_gap + num), (255, 255, 255))
draw = ImageDraw.Draw(image)
font_size = round(num / 4 * 3)
font = ImageFont.truetype('Roboto-Bold.ttf', font_size)
for i in range(image_shape[0]):
    for j in range(image_shape[1]):
        x1 = border + j * (square + grid_gap)
        y1 = border + i * (square + grid_gap)
        x2 = border + (j + 1) * square + j * grid_gap
        y2 = border + (i + 1) * square + i * grid_gap
        if borders:
            if i * image_shape[1] + j == str(n).find('7'):
                draw.rectangle((x1, y1, x2, y2), fill=colors[n[i * image_shape[1] + j]][::-1],
outline=(0, 255, 0), width=10)
            else:

```

```

        draw.rectangle((x1, y1, x2, y2), fill=colors[n[i * image_shape[1] + j]][::-1],
outline=(0, 0, 0), width=10)
    else:
        draw.rectangle((x1, y1, x2, y2), fill=colors[n[i * image_shape[1] + j]][::-1])
        draw.text((image.size[0] // 2, image.size[1] - border - num // 2), f'{w} x {h}', fill='black',
font=font, anchor='mm')
    if borders:
        image.save(f'images/{user_id}.png')
        return True
    elif pixelate:
        image.save(f'images/pixelate {pixelate} {user_id}.png')
        return f'{h} {w} {await convert_to(n, 7, 64)}'
    else:
        cfcode = await db.get_cfcode_by_id(f'{h} {w} {await convert_to(n, 7, 64)}')
        image.save(f'images/{cfcode["cfcode_num"]}.png')
        return cfcode["cfcode_num"]

```

ПРИЛОЖЕНИЕ 3

Полный код файла database.py:

```
import sqlite3 as sq

db = sq.connect('cfcode.db')
cursor = db.cursor()

# Удаление таблиц из базы данных
async def delete_tables():
    cursor.execute("DROP TABLE IF EXISTS users")
    cursor.execute("DROP TABLE IF EXISTS cfcodes")
    db.commit()

# Создание таблиц в базе данных
async def create_tables():
    # Создание таблицы пользователей
    cursor.execute("CREATE TABLE IF NOT EXISTS users("
        "user_id INTEGER PRIMARY_KEY, " # Telegram id пользователя
        "step TEXT, " # Шаг, на котором находится пользователь
        "page INTEGER, " # Страница CF-кодов, которую пользователь просматривает
        "text TEXT, " # Текст, который будет сохранён в CF-коде
        "cfcode_id TEXT, " # Id CF-кода, с которым взаимодействует пользователь
        "cfcode_num INTEGER)") # Количество CF-кодов, созданных пользователем

    # Создание таблицы CF-кодов
    cursor.execute("CREATE TABLE IF NOT EXISTS cfcodes("
        "cfcode_id TEXT PRIMARY KEY, " # Id CF-кода
        "cfcode_num INTEGER, " # Номер CF-кода, под которым он будет сохранен как
изображение
        "pixelate INTEGER, " # Уровень пикселизации изображения
        "owner_id INTEGER, " # Telegram id пользователя, создавшего CF-код
        "text TEXT, " # Текст CF-кода
        "date TEXT, " # Дата и время создания CF-кода
        "views INTEGER)") # Количество использований CF-кода
    db.commit()

# Добавление нового пользователя в базу данных
async def add_new_user(user_id):
    cursor.execute(f"INSERT INTO users (user_id, step, page, text, cfcode_id, cfcode_num)
VALUES ({user_id}, ", 1, ", ", 0)")
```

```
db.commit()
```

```
# Поиск пользователя в базе данных по его telegram id
```

```
async def get_user_by_id(user_id):  
    user = cursor.execute(f"SELECT * FROM users WHERE user_id == {user_id}").fetchone()  
    if user is None:  
        return False  
    res = {'user_id': None, 'step': None, 'page': None, 'text': None, 'cfcode_id': None, 'cfcode_num':  
None}  
    res_keys = list(res.keys())  
    for i in range(len(user)):  
        res[res_keys[i]] = user[i]  
    return res
```

```
# Изменение шага, на котором находится пользователь
```

```
async def update_user_step(user_id, step):  
    cursor.execute(f"UPDATE users SET step = '{step}' WHERE user_id == {user_id}")  
    db.commit()
```

```
# Изменение страницы CF-кодов пользователя
```

```
async def update_user_page(user_id, page):  
    cursor.execute(f"UPDATE users SET page = {page} WHERE user_id == {user_id}")  
    db.commit()
```

```
# Изменение текста, который будет сохранен в CF-коде
```

```
async def update_user_text(user_id, text):  
    cursor.execute(f"UPDATE users SET text = '{text}' WHERE user_id == {user_id}")  
    db.commit()
```

```
# Изменение id CF-кода, с которым взаимодействует пользователь
```

```
async def update_user_cfcode_id(user_id, cfcode_id):  
    cursor.execute(f"UPDATE users SET cfcode_id = '{cfcode_id}' WHERE user_id ==  
{user_id}")  
    db.commit()
```

```
# Добавление нового CF-кода в базу данных
```

```
async def add_new_cfcode(owner_id, cfcode_id, pixelate, text, date):  
    cfcode_num = cursor.execute(f"SELECT MAX(cfcode_num) FROM cfcodes").fetchone()[0]
```



```

if cfcode_num is None:
    cfcode_num = 0
    cursor.execute(f"INSERT INTO cfcodes (cfcode_id, cfcode_num, pixelate, owner_id, text,
date, views) VALUES ('{cfcode_id}', {int(cfcode_num) + 1}, {pixelate}, {owner_id}, '{text}',
'{date}', 0)")
    db.commit()

# Увеличение количества CF-кодов пользователя
async def increase_cfcode_num(user_id):
    cfcode_num = cursor.execute(f"SELECT cfcode_num FROM users WHERE user_id ==
{user_id}").fetchone()
    cursor.execute(f"UPDATE users SET cfcode_num = {int(cfcode_num[0]) + 1} WHERE
user_id == {user_id}")
    db.commit()

# Поиск CF-кода по его id
async def get_cfcode_by_id(cfcode_id):
    cfcode = cursor.execute(f"SELECT * FROM cfcodes WHERE cfcode_id ==
'{cfcode_id}'").fetchone()
    #print(cfcode)
    if cfcode is None:
        return None
    res = {'cfcode_id': None, 'cfcode_num': None, 'pixelate': None, 'owner_id': None, 'text': None,
'date': None, 'views': None}
    res_keys = ['cfcode_id', 'cfcode_num', 'pixelate', 'owner_id', 'text', 'date', 'views']
    for i in range(len(cfcode)):
        res[res_keys[i]] = cfcode[i]
    return res

# Поиск всех CF-кодов пользователя в базе данных по его telegram id
async def get_users_cfcodes(user_id):
    cfcodes = cursor.execute(f"SELECT * FROM cfcodes WHERE owner_id ==
{user_id}").fetchall()
    if cfcodes == []:
        return None
    res = []
    for cfcode in cfcodes:
        res.append({'cfcode_id': cfcode[0], 'cfcode_num': cfcode[1], 'owner_id': cfcode[3], 'text':
cfcode[4], 'date': cfcode[5], 'views': cfcode[6]})
    return res

```

```

# Поиск пикселизированных CF-кодов пользователя
async def get_user_pixelate_cfcode(user_id, pixelate):
    cfcode = cursor.execute(f'SELECT * FROM cfcodes WHERE owner_id == {user_id} AND
pixelate == {pixelate}').fetchone()
    if cfcode is None:
        return False
    res = {'cfcode_id': cfcode[0], 'cfcode_num': cfcode[1], 'pixelate': cfcode[2], 'owner_id':
cfcode[3], 'text': cfcode[4], 'date': cfcode[5], 'views': cfcode[6]}
    return res

# Увеличение количества просмотров CF-кода
async def view_cfcode(cfcode_id):
    views = cursor.execute(f'SELECT views FROM cfcodes WHERE cfcode_id ==
'{cfcode_id}').fetchone()[0]
    cursor.execute(f'UPDATE cfcodes SET views = {views + 1} WHERE cfcode_id ==
'{cfcode_id}')
    db.commit()

# Изменение текста CF-кода
async def update_cfcode_text(cfcode_id, text):
    cursor.execute(f'UPDATE cfcodes SET text = '{text}' WHERE cfcode_id == '{cfcode_id}')
    db.commit()

# Удаление CF-кода из базы данных
async def delete_cfcode(user_id, cfcode_id):
    cursor.execute(f'DELETE FROM cfcodes WHERE cfcode_id == '{cfcode_id}')
    user = await get_user_by_id(user_id)
    cursor.execute(f'UPDATE users SET cfcode_num == {user["cfcode_num"] - 1}')
    db.commit()

```

ПРИЛОЖЕНИЕ 4

Полный код файла tgbot.py:

```
from aiogram import Bot, Dispatcher, executor, types
from aiogram.types import ReplyKeyboardMarkup
from PIL import Image
import requests
import datetime
import random
import io
import os
import cfcode as cf
import database as db

# Токен телеграм бота
bot = Bot('BOT_TOKEN')
uri = f'https://api.telegram.org/botBOT_TOKEN/getFile?file_id='
uri_img = f'https://api.telegram.org/file/botBOT_TOKEN/'
dp = Dispatcher(bot)

# Функция для создания ReplyKeyboardMarkup
def make_keyboard(arr):
    keyboard = ReplyKeyboardMarkup(resize_keyboard=True)
    for elem in arr:
        keyboard.add(elem)
    return keyboard

# Функция, вызываемая при запуске бота
async def on_startup(_):
    #await db.delete_tables()
    await db.create_tables()

# Кнопка /start
@dp.message_handler(commands=['start'])
async def cmd_start(message: types.Message):
    await message.answer(f'Привет, это бот, с помощью которого ты сможешь считывать и генерировать CF-кода\n'
                        f'Данный проект является лишь небольшим прототипом, основной концепции, в соответствии с чем '
                        f'далек от идеала и имеет возможность претерпевать различные изменения\nПри использовании, '
```

```

        f'помни, что я всего на всего бедный школьник с незаурядными знаниями в
программировании, '
        f'будь вежлив и милосерден\nВыбери действие, чтобы продолжить',
        reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-коды']))

# Обработка получаемых изображений
@dp.message_handler(content_types=['photo'])
async def cmd_photo(message: types.Message):
    # Получение информации о пользователе из базы данных
    user = await db.get_user_by_id(message.chat.id)
    # Занесение пользователя в базу данных, если его там нет
    if user is False:
        await db.add_new_user(message.chat.id)
        user = await db.get_user_by_id(message.chat.id)

    # Считать CF-код на изображении
    if user['step'].split()[2] == ['read', 'image'] and len(user['step'].split()) == 4:
        res = requests.get(uri + message.photo[-1].file_id)
        img_path = res.json()['result']['file_path']
        img = requests.get(uri_img + img_path)
        img = Image.open(io.BytesIO(img.content))
        res = await cf.detect_code(img, int(user['step'].split()[2]), int(user['step'].split()[3]))
        if res is None:
            await message.answer('Не удалось считать CF-код, попробуй сделать другую
фотографию', reply_markup=make_keyboard(['Меню', 'Назад']))
        else:
            result = await db.get_cfcode_by_id(f'{res[1]} {res[2]} {res[0]}')
            if result is not None:
                await db.view_cfcode(f'{res[1]} {res[2]} {res[0]}')
                await message.answer(result['text'], reply_markup=make_keyboard(['Меню', 'Назад']))
            return
            await message.answer('Не удалось найти CF-код в базе данных',
reply_markup=make_keyboard(['Меню', 'Назад']))

    # Загрузить готовое изображение
    elif user['step'].split()[2] == ['import', 'image'] and len(user['step'].split()) == 4:
        res = requests.get(uri + message.photo[-1].file_id)
        img_path = res.json()['result']['file_path']
        img = requests.get(uri_img + img_path)
        img = Image.open(io.BytesIO(img.content))
        res = await cf.detect_code(img, int(user['step'].split()[2]), int(user['step'].split()[3]))
        if res is None:

```

```

        await message.answer('Не удалось считать CF-код, попробуй отправить другую
фотографию')
    elif await db.get_cfcode_by_id(f'{res[1]} {res[2]} {res[0]}') is None:
        offset = datetime.timezone(datetime.timedelta(hours=3))
        date = datetime.datetime.now(offset)
        await db.add_new_cfcode(message.chat.id, f'{res[1]} {res[2]} {res[0]}', 0, user['text'],
str(date)[:19])
        await db.increase_cfcode_num(message.chat.id)
        cfcode_num = await cf.generate_code(res[0], res[1], res[2], False)
        await db.update_user_step(message.chat.id, "")
        await db.update_user_text(message.chat.id, "")
        photo = open(f'images/{cfcode_num}.png', 'rb')
        await bot.send_photo(message.chat.id, photo=photo, caption='CF-код добавлен,
убедитесь в правильности считывания', reply_markup=make_keyboard(['Считать CF-код',
'Мои CF-кода']))
        photo.close()
    else:
        await message.answer('Данный CF-код уже использован, попробуй другой',
reply_markup=make_keyboard(['Меню', 'Назад']))

# Пикселизировать изображение
elif user['step'] == 'pixelate image':
    await db.update_user_step(message.chat.id, 'pixelate choose')
    res = requests.get(uri + message.photo[-1].file_id)
    img_path = res.json()['result']['file_path']
    img = requests.get(uri_img + img_path)
    img = Image.open(io.BytesIO(img.content))
    img16 = await cf.pixelate(img, 16)
    img24 = await cf.pixelate(img, 24)
    img32 = await cf.pixelate(img, 32)
    await cf.generate_code(img16[0], img16[1], img16[2], False, user_id=message.chat.id,
pixelate=16)
    await cf.generate_code(img24[0], img24[1], img24[2], False, user_id=message.chat.id,
pixelate=24)
    await cf.generate_code(img32[0], img32[1], img32[2], False, user_id=message.chat.id,
pixelate=32)
    photo16 = open(f'images/pixelate {16} {message.chat.id}.png', 'rb')
    if await db.get_cfcode_by_id(f'{message.chat.id} {img16[1]} {img16[2]} {img16[0]}') is
not None:
        await db.delete_cfcode(message.chat.id, f'{message.chat.id} {img16[1]} {img16[2]}
{img16[0]}')
        await db.add_new_cfcode(message.chat.id, f'{message.chat.id} {img16[1]} {img16[2]}
{img16[0]}', 16, user['text'], '0')

```

```

        await bot.send_photo(message.chat.id, photo=photo16, caption='1 уровень пикселизации',
reply_markup=make_keyboard(['1', '2', '3']))
        photo24 = open(f'images//pixelate {24} {message.chat.id}.png', 'rb')
        if await db.get_cfcode_by_id(f'{message.chat.id} {img24[1]} {img24[2]} {img24[0]}') is
not None:
            await db.delete_cfcode(message.chat.id, f'{message.chat.id} {img24[1]} {img24[2]}
{img24[0]}')
            await db.add_new_cfcode(message.chat.id, f'{message.chat.id} {img24[1]} {img24[2]}
{img24[0]}', 24, user['text'], '0')
            await bot.send_photo(message.chat.id, photo=photo24, caption='2 уровень пикселизации',
reply_markup=make_keyboard(['1', '2', '3']))
            photo32 = open(f'images//pixelate {32} {message.chat.id}.png', 'rb')
            if await db.get_cfcode_by_id(f'{message.chat.id} {img32[1]} {img32[2]} {img32[0]}') is
not None:
                await db.delete_cfcode(message.chat.id, f'{message.chat.id} {img32[1]} {img32[2]}
{img32[0]}')
                await db.add_new_cfcode(message.chat.id, f'{message.chat.id} {img32[1]} {img32[2]}
{img32[0]}', 32, user['text'], '0')
                await bot.send_photo(message.chat.id, photo=photo32, caption='3 уровень пикселизации',
reply_markup=make_keyboard(['1', '2', '3']))
            await message.answer('Выбери уровень пикселизации',
reply_markup=make_keyboard(['1', '2', '3', 'Меню', 'Назад']))
            photo16.close()
            photo24.close()
            photo32.close()

    else:
        await message.answer('Неизвестная команда', reply_markup=make_keyboard(['Меню',
'Назад']))

# Функция отображения моих CF-кодов
async def pages(message):
    cfcodes = await db.get_users_cfcodes(message.chat.id)
    user = await db.get_user_by_id(message.chat.id)
    if cfcodes is None:
        await message.answer('У тебя еще нет CF-кодов, но ты можешь их сгенерировать',
reply_markup=make_keyboard(
            ['Сгенерировать случайный', 'Загрузить изображение', 'Создать CF-код', 'Назад']))
    else:
        for i in range(len(cfcodes[(user['page'] - 1) * 5: user['page'] * 5])):
            photo = open(f'images//{cfcodes[(user["page"] - 1) * 5 + i]["cfcode_num"]}.png', 'rb')
            await bot.send_photo(message.chat.id, photo=photo, caption=f'{(user["page"] - 1) * 5 + i
+ 1}. Текст: {cfcodes[(user["page"] - 1) * 5 + i]["text"]}\nИспользований:

```

```
{cfcodes[(user["page"] - 1) * 5 + i][["views"]]\nДата создания: {cfcodes[(user["page"] - 1) * 5 + i][["date"]]}')
    photo.close()
```

```
    if len(cfcodes) > user['page'] * 5 and user['page'] != 1:
        await message.answer(f'Страница {user["page"]} из {(len(cfcodes) + 4) // 5}',
reply_markup=make_keyboard(['Следующая страница', 'Предыдущая страница',
'Редактировать CF-код', 'Создать новый', 'Назад']))
    elif user['page'] != 1:
        await message.answer(f'Страница {user["page"]} из {(len(cfcodes) + 4) // 5}',
reply_markup=make_keyboard(['Предыдущая страница', 'Редактировать CF-код', 'Создать
новый', 'Назад']))
    elif len(cfcodes) > user['page'] * 5:
        await message.answer(f'Страница {user["page"]} из {(len(cfcodes) + 4) // 5}',
reply_markup=make_keyboard(['Следующая страница', 'Редактировать CF-код', 'Создать
новый', 'Назад']))
    else:
        await message.answer(f'Страница {user["page"]} из {(len(cfcodes) + 4) // 5}',
reply_markup=make_keyboard(['Редактировать CF-код', 'Создать новый', 'Назад']))
```

```
commands = ['следующая страница', 'предыдущая страница', 'редактировать CF-код',
'создать новый', 'считать CF-код', 'мои CF-кода', 'меню', 'изменить текст', 'удалить CF-код']
```

```
# Обработка получаемых сообщений
@dp.message_handler()
async def cmd_text(message: types.Message):
    # Получение информации о пользователе из базы данных
    user = await db.get_user_by_id(message.chat.id)
    # Занесение пользователя в базу данных, если его там нет
    if user is False:
        await db.add_new_user(message.chat.id)
        user = await db.get_user_by_id(message.chat.id)

    # Кнопка Меню
    if message.text.lower() == 'меню':
        # Обнуление всех переменных, связанных с работой бота
        await db.update_user_step(message.chat.id, "")
        await db.update_user_text(message.chat.id, "")
        await db.update_user_page(message.chat.id, 0)
        await db.update_user_cfcode_id(message.chat.id, "")
        await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))
```

```

# Кнопка Назад
elif message.text.lower() == 'назад':
    if user['step'] == "":
        await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))
    elif user['step'] in ['read image size', 'page', 'edit']:
        await db.update_user_step(message.chat.id, "")
        await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))
    elif user['step'].split()[:2] == ['read', 'image']:
        await db.update_user_step(message.chat.id, 'read image size')
        await message.answer('Введи размер CF-кода, указанный под ним, разделяя числа
пробелом', reply_markup=make_keyboard(['Меню', 'Назад']))
    elif user['step'] == 'read image size':
        await db.update_user_step(message.chat.id, "")
        await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))
    elif user['step'] == 'image choose' or user['step'] == 'text':
        await db.update_user_step(message.chat.id, "")
        await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(
    ['Сгенерировать случайный', 'Загрузить изображение', 'Создать CF-код', 'Меню',
'Назад']))
    elif user['step'] == 'pixelate choose':
        await db.update_user_step(message.chat.id, 'pixelate image')
        await message.answer('Отправь изображение для пикселизации',
reply_markup=make_keyboard(['Меню', 'Назад']))
    elif user['step'] == 'generate random':
        await db.update_user_page(message.chat.id, 1)
        await db.update_user_step(message.chat.id, 'page')
        await pages(message)
    elif user['step'] == 'pixelate image':
        await db.update_user_step(message.chat.id, 'pixelate text')
        await message.answer('Отправь текст, который будет отображаться при сканировании
CF-кода', reply_markup=make_keyboard(['Меню', 'Назад']))
    elif user['step'] == 'pixelate text':
        await db.update_user_step(message.chat.id, 'image choose')
        await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Загрузить готовое', 'Пикселизировать', 'Назад']))
    elif user['step'] == 'editing':
        await db.update_user_step(message.chat.id, 'edit')
        await message.answer('Укажите номер CF-кода для редактирования (номер CF-кода
указан под фотографией)')

```



```

elif user['step'] == 'delete':
    cfcode = await db.get_cfcode_by_id(user['cfcode_id'])
    await db.update_user_step(message.chat.id, 'editing')
    photo = open(f'images/{cfcode["cfcode_num"]}.png', 'rb')
    await bot.send_photo(message.chat.id, photo=photo, caption='Выберите действие,
чтобы продолжить', reply_markup=make_keyboard(['Изменить текст', 'Удалить CF-код',
'Меню', 'Назад']))
    photo.close()
elif user['step'].split()[:2] == ['import', 'image']:
    await db.update_user_step(message.chat.id, 'image choose')
    await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Загрузить готовое', 'Пикселизировать', 'Назад']))
elif user['step'] == 'import text':
    await db.update_user_step(message.chat.id, 'image choose')
    await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Загрузить готовое', 'Пикселизировать', 'Назад']))

# Кнопка Считать CF-код
elif message.text.lower() == 'считать cf-код':
    await db.update_user_step(message.chat.id, 'read image size')
    await message.answer('Введи размер CF-кода, указанный под ним, разделяя числа
пробелом', reply_markup=make_keyboard(['Меню', 'Назад']))

# Ввод размера CF-кода
elif user['step'] == 'read image size' and message.text.lower() not in commands:
    if message.text.count(' ') == 1 and message.text.split()[0].isnumeric() and
message.text.split()[1].isnumeric():
        await db.update_user_step(message.chat.id, f'read image {message.text}')
        await message.answer('Отправь изображение с CF-кодом, предварительно его
обрезав по белой окантовке', reply_markup=make_keyboard(['Меню', 'Назад']))
    else:
        await message.answer('Неверно указан размер, попробуй снова',
reply_markup=make_keyboard(['Меню', 'Назад']))

# Кнопка Мои CF-кода
elif message.text.lower() == 'мои cf-кода':
    await db.update_user_page(message.chat.id, 1)
    await db.update_user_step(message.chat.id, 'page')
    await pages(message)

# Кнопка Следующая страница
elif user['step'] == 'page' and message.text.lower() == 'следующая страница' and
user['cfcode_num'] >= user['page'] * 5:

```

```

    await db.update_user_page(message.chat.id, user['page'] + 1)
    await pages(message)

# Кнопка Предыдущая страница
elif user['step'] == 'page' and message.text.lower() == 'предыдущая страница' and user['page']
- 1 != 0:
    await db.update_user_page(message.chat.id, user['page'] - 1)
    await pages(message)

# Кнопка Редактировать CF-код
elif message.text.lower() == 'редактировать cf-код':
    await db.update_user_step(message.chat.id, 'edit')
    await message.answer('Укажи номер CF-кода для редактирования (номер CF-кода
указан под фотографией)')

# Ввод номера CF-кода для редактирования
elif user['step'] == 'edit' and message.text.lower() not in commands:
    if not (message.text.isnumeric()) or int(message.text) < 1 or int(message.text) >
user['cfcode_num']:
        await message.answer('Некорректный номер CF-кода, попробуй снова')
    else:
        cfcode = await db.get_users_cfcodes(message.chat.id)
        cfcode = cfcode[int(message.text) - 1]
        await db.update_user_cfcode_id(message.chat.id, cfcode['cfcode_id'])
        await db.update_user_step(message.chat.id, 'editing')
        photo = open(f'images/{cfcode["cfcode_num"]}.png', 'rb')
        await bot.send_photo(message.chat.id, photo=photo, caption='Выбери действие, чтобы
продолжить', reply_markup=make_keyboard(['Изменить текст', 'Удалить CF-код', 'Меню',
'Назад']))
        photo.close()

# Кнопка Изменить текст
elif user['step'] == 'editing' and message.text.lower() == 'изменить текст':
    await db.update_user_step(message.chat.id, 'editing text')
    await message.answer('Отправь новый текст, который будет отображаться при
сканировании CF-кода')

# Изменение текста CF-кода
elif user['step'] == 'editing text' and message.text.lower() not in commands:
    await db.update_cfcode_text(user['cfcode_id'], message.text)
    await db.update_user_step(message.chat.id, '')
    await message.answer('Текст успешно изменен', reply_markup=make_keyboard(['Считать
CF-код', 'Мои CF-кода']))

```

```

# Кнопка Удалить CF-код
elif user['step'] == 'editing' and message.text.lower() == 'удалить cf-код':
    await db.update_user_step(message.chat.id, 'delete')
    await message.answer('Ты уверен, что хочешь удалить CF-код?',
reply_markup=make_keyboard(['Да', 'Нет', 'Меню', 'Назад']))

# Удаление CF-кода
elif user['step'] == 'delete' and message.text.lower() == 'да':
    cfcode = await db.get_cfcode_by_id(user['cfcode_id'])
    await db.delete_cfcode(message.chat.id, user['cfcode_id'])
    os.remove(f'images/{cfcode["cfcode_num"]}.png')
    await db.update_user_step(message.chat.id, '')
    await message.answer('CF-код удален', reply_markup=make_keyboard(['Считать CF-код',
'Mои CF-кода']))

elif user['step'] == 'delete' and message.text.lower() == 'нет':
    await db.update_user_step(message.chat.id, '')
    await message.answer('Выберите действие, чтобы продолжить',
reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))

# Кнопка Создать новый
elif message.text.lower() == 'создать новый':
    await db.update_user_step(message.chat.id, '')
    await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Сгенерировать случайный', 'Загрузить изображение',
'Создать CF-код', 'Меню', 'Назад']))

# Кнопка Сгенерировать случайный
elif message.text.lower() == 'сгенерировать случайный':
    await db.update_user_step(message.chat.id, 'generate random')
    await message.answer('Отправь текст, который будет отображаться при сканировании
CF-кода')

# Генерация случайного CF-кода
elif user['step'] == 'generate random' and message.text.lower() not in commands:
    w = random.randint(3, 10)
    h = random.randint(2, w - 1)
    n = ''.join(random.choices('0123456', k=h * w))
    while await db.get_cfcode_by_id(n) is not None:
        n = ''.join(random.choices('0123456', k=h * w))
    offset = datetime.timezone(datetime.timedelta(hours=3))
    date = datetime.datetime.now(offset)
    await db.add_new_cfcode(message.chat.id, f'{h} {w} {await cf.convert_to(n, 7, 64)}', 0,
message.text, str(date)[:19])

```

```

await db.increase_cfcode_num(message.chat.id)
image = await cf.generate_code(await cf.convert_to(n, 7, 64), h, w, False, None, False)
photo = open(f'images/{image}.png', 'rb')
await db.update_user_step(message.chat.id, "")
await bot.send_photo(message.chat.id, photo=photo, caption='Твой сгенерированный
CF-код', reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))
photo.close()

# Кнопка Загрузить изображение
elif message.text.lower() == 'загрузить изображение':
    await db.update_user_step(message.chat.id, 'image choose')
    await message.answer('Выбери действие, чтобы продолжить',
reply_markup=make_keyboard(['Загрузить готовое', 'Пикселизировать', 'Назад']))

# Кнопка Загрузить готовое
elif user['step'] == 'image choose' and message.text.lower() == 'загрузить готовое':
    await db.update_user_step(message.chat.id, 'import text')
    await message.answer('Отправь текст, который будет отображаться при сканировании
CF-кода', reply_markup=make_keyboard(['Назад']))

# Ввод текста CF-кода
elif user['step'] == 'import text' and message.text.lower() not in commands:
    await db.update_user_step(message.chat.id, 'import image')
    await db.update_user_text(message.chat.id, message.text)
    await message.answer('Введи размер CF-кода, указанный под ним, разделяя числа
пробелом')

# Ввод размера CF-кода
elif user['step'] == 'import image' and message.text.lower() not in commands:
    if message.text.count(' ') == 1 and message.text.split()[0].isnumeric() and
message.text.split()[1].isnumeric():
        await db.update_user_step(message.chat.id, f'import image {message.text}')
        await message.answer('Отправь готовое изображение, содержащее CF-код')
    else:
        await message.answer('Неверно указан размер, попробуй снова',
reply_markup=make_keyboard(['Назад']))

# Кнопка Пикселизировать
elif user['step'] == 'image choose' and message.text.lower() == 'пикселизировать':
    await db.update_user_step(message.chat.id, 'pixelate text')
    await message.answer('С помощью этой функции ты сможешь создать CF-код
посредством пикселизации своего изображения\nОтправь текст, который будет
отображаться при сканировании CF-кода', reply_markup=make_keyboard(['Меню', 'Назад']))

```

```

# Ввод текста CF-кода для пикселизации
elif user['step'] == 'pixelate text' and message.text.lower() not in commands:
    await db.update_user_step(message.chat.id, 'pixelate image')
    await db.update_user_text(message.chat.id, message.text)
    await message.answer('Отправь изображение для пикселизации',
reply_markup=make_keyboard(['Меню', 'Назад']))

# Выбор нужного пикселизированного изображения
elif user['step'] == 'pixelate choose' and message.text.lower() not in commands:
    if message.text in ['1', '2', '3']:
        await db.update_user_step(message.chat.id, "")
        await db.increase_cfcode_num(message.chat.id)
        cfcode16 = await db.get_user_pixelate_cfcode(message.chat.id, 16)
        cfcode24 = await db.get_user_pixelate_cfcode(message.chat.id, 24)
        cfcode32 = await db.get_user_pixelate_cfcode(message.chat.id, 32)
        await db.delete_cfcode(message.chat.id, cfcode16['cfcode_id'])
        await db.delete_cfcode(message.chat.id, cfcode24['cfcode_id'])
        await db.delete_cfcode(message.chat.id, cfcode32['cfcode_id'])
        os.remove(f'images//pixelate 16 {message.chat.id}.png')
        os.remove(f'images//pixelate 24 {message.chat.id}.png')
        os.remove(f'images//pixelate 32 {message.chat.id}.png')
        offset = datetime.timezone(datetime.timedelta(hours=3))
        date = datetime.datetime.now(offset)
        if message.text == '1':
            if await db.get_cfcode_by_id(' '.join(cfcode16['cfcode_id'].split()[1:])) is None:
                await db.add_new_cfcode(message.chat.id, ' '.join(cfcode16['cfcode_id'].split()[1:]),
0, user['text'], str(date)[:19])
                image = await cf.generate_code(cfcode16['cfcode_id'].split()[-1],
int(cfcode16['cfcode_id'].split()[-3]), int(cfcode16['cfcode_id'].split()[-2]), False, None, False)
                await db.update_user_step(message.chat.id, "")
                photo16 = open(f'images//{image}.png', 'rb')
                await bot.send_photo(message.chat.id, photo=photo16, caption='Твой
пикселизированный CF-код', reply_markup=make_keyboard(['Считать CF-код', 'Мои
CF-кода']))
                photo16.close()
            else:
                await db.update_user_step(message.chat.id, 'pixelate image')
                await message.answer('Данный код уже использован, отправь другое
изображение', reply_markup=make_keyboard(['Меню', 'Назад']))
        elif message.text == '2':
            if await db.get_cfcode_by_id(' '.join(cfcode24['cfcode_id'].split()[1:])) is None:
                await db.add_new_cfcode(message.chat.id, ' '.join(cfcode24['cfcode_id'].split()[1:]),
0, user['text'], str(date)[:19])

```

```

        image = await cf.generate_code(cfcode24['cfcode_id'].split()[-1],
int(cfcode24['cfcode_id'].split()[-3]), int(cfcode24['cfcode_id'].split()[-2]), False, None, False)
        await db.update_user_step(message.chat.id, "")
        photo24 = open(f'images/{image}.png', 'rb')
        await bot.send_photo(message.chat.id, photo=photo24, caption='Твой
пикселизированный CF-код', reply_markup=make_keyboard(['Считать CF-код', 'Мои
CF-кода']))
        photo24.close()
    else:
        await db.update_user_step(message.chat.id, 'pixelate image')
        await message.answer('Данный код уже использован, отправь другое
изображение', reply_markup=make_keyboard(['Меню', 'Назад']))
    else:
        if await db.get_cfcode_by_id(' '.join(cfcode32['cfcode_id'].split()[1:])) is None:
            await db.add_new_cfcode(message.chat.id, ' '.join(cfcode32['cfcode_id'].split()[1:]),
0, user['text'], str(date)[:19])
            image = await cf.generate_code(cfcode32['cfcode_id'].split()[-1],
int(cfcode32['cfcode_id'].split()[-3]), int(cfcode32['cfcode_id'].split()[-2]), False, None, False)
            await db.update_user_step(message.chat.id, "")
            photo32 = open(f'images/{image}.png', 'rb')
            await bot.send_photo(message.chat.id, photo=photo32, caption='Твой
пикселизированный CF-код', reply_markup=make_keyboard(['Считать CF-код', 'Мои
CF-кода']))
            photo32.close()
        else:
            await db.update_user_step(message.chat.id, 'pixelate image')
            await message.answer('Данный код уже использован, отправь другое
изображение', reply_markup=make_keyboard(['Меню', 'Назад']))
    else:
        await message.answer('Неверно указан номер, попробуй снова',
reply_markup=make_keyboard(['1', '2', '3', 'Меню', 'Назад']))

# Кнопка Создать CF-код
elif message.text.lower() == 'создать cf-код':
    await db.update_user_step(message.chat.id, 'text')
    await message.answer('С помощью этой функции ты сможешь создать небольшой
CF-код, учти, что процесс этот кропотливый и в случае нетерпеливости ты можешь
воспользоваться функцией \'Загрузить изображение\', чтобы продолжить отправь текст,
который будет отображаться при сканировании CF-кода')

# Ввод текста CF-кода
elif user['step'] == 'text' and message.text.lower() not in commands:
    await db.update_user_step(message.chat.id, 'draw code columns')
    await db.update_user_text(message.chat.id, message.text)

```

```

await message.answer('Введи количество столбцов CF-кода от 2 до 5')

# Ввод количества столбцов
elif user['step'] == 'draw code columns' and message.text.lower() not in commands:
    if not (message.text.isnumeric()) or int(message.text) < 2 or int(message.text) > 5:
        await message.answer('Некорректное количество столбцов, попробуй снова')
    else:
        await db.update_user_step(message.chat.id, 'draw code lines')
        await db.update_user_cfcode_id(message.chat.id, message.text)
        await message.answer(f'Введи количество строк от 1 до 5')

# Ввод количества строк
elif user['step'] == 'draw code lines' and message.text.lower() not in commands:
    if not (message.text.isnumeric()) or int(message.text) < 1 or int(message.text) > 5:
        await message.answer('Некорректное количество строк, попробуй снова')
    else:
        await db.update_user_step(message.chat.id, 'draw code')
        await db.update_user_cfcode_id(message.chat.id, f'{user["cfcode_id"]} {message.text}'
        {'7' * int(user["cfcode_id"]) * int(message.text)})
        await cf.generate_code('7' * int(user["cfcode_id"]) * int(message.text), int(message.text),
int(user["cfcode_id"]), True, user_id=message.chat.id)
        photo = open(f'images/{message.chat.id}.png', 'rb')
        await bot.send_photo(message.chat.id, photo=photo, caption='Выбери цвет квадрата',
reply_markup=make_keyboard(['Черный', 'Красный', 'Желтый', 'Зеленый', 'Голубой', 'Синий',
'Розовый', 'Меню']))
        photo.close()

# Выбор цвета квадрата CF-кода
elif user['step'] == 'draw code' and message.text.lower() not in commands:
    if message.text not in ['Черный', 'Красный', 'Желтый', 'Зеленый', 'Голубой', 'Синий',
'Розовый']:
        await message.answer('Некорректный цвет, попробуй снова')
    else:
        colors = {'Черный': 0, 'Красный': 1, 'Желтый': 2, 'Зеленый': 3, 'Голубой': 4, 'Синий': 5,
'Розовый': 6}
        n = str(user["cfcode_id"]).split()[2]
        i = n.find('7')
        n = n[:i] + str(colors[message.text]) + n[i + 1:]
        await db.update_user_cfcode_id(message.chat.id, f'{str(user["cfcode_id"]).split()[0]}
{str(user["cfcode_id"]).split()[1]} {n}')
        if n.count('7') == 0:
            if await db.get_cfcode_by_id(f'{user["cfcode_id"].split()[1]}
{user["cfcode_id"].split()[0]} {await cf.convert_to(n, 7, 64)}') is None:
                offset = datetime.timezone(datetime.timedelta(hours=3))

```

```

        date = datetime.datetime.now(offset)
        await db.add_new_cfcode(message.chat.id, f'{int(str(user['cfcode_id']).split()[1]))
        {int(str(user['cfcode_id']).split()[0])) {await cf.convert_to(n, 7, 64)}", 0, user['text'],
        str(date)[:19])
        await db.increase_cfcode_num(message.chat.id)
        image = await cf.generate_code(n, int(str(user['cfcode_id']).split()[1]),
        int(str(user['cfcode_id']).split()[0]), False, user_id=message.chat.id, convert=False)
        photo = open(f'images/{image}.png', 'rb')
        await bot.send_photo(message.chat.id, photo=photo, caption='CF-код добавлен',
        reply_markup=make_keyboard(['Считать CF-код', 'Мои CF-кода']))
        photo.close()
    else:
        await message.answer('Данный CF-код уже использован, попробуй другой',
        reply_markup=make_keyboard(['Меню', 'Назад']))
    else:
        await cf.generate_code(n, int(str(user['cfcode_id']).split()[1]),
        int(str(user['cfcode_id']).split()[0]), True, user_id=message.chat.id)
        photo = open(f'images/{message.chat.id}.png', 'rb')
        await bot.send_photo(message.chat.id, photo=photo, caption='Выбери цвет квадрата',
        reply_markup=make_keyboard(['Черный', 'Красный', 'Желтый', 'Зеленый', 'Голубой', 'Синий',
        'Розовый', 'Меню']))
        photo.close()

    else:
        await message.answer(f'Неизвестная команда, попробуй снова')

if __name__ == '__main__':
    executor.start_polling(dp, on_startup=on_startup, skip_updates=True)

```


БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Глория, Буэно Гарсия. Обработка изображений с помощью OpenCV, 2016.
2. Max E. Vizcarra Melgar, Alexandre Zaghetto, Bruno Macchiavello, Anderson C. A.

Nascimento CQR CODES: COLORED QUICK-RESPONSE CODES

3. <https://habr.com/ru/companies/skillfactory/articles/528320/>
4. <https://trends.rbc.ru/trends/industry/6189517c9a79475deb5dbf9a>